

ROTEIRO DE FALAS · USO DO INSTRUTOR

GIT, GITLAB E IA APLICADA
À ENGENHARIA DE SOFTWARE

GIT · IA · DOCS

Ronivaldo D. Andrade · Marinha do Brasil · 01/10/2026, das 10h às 11h

Como usar. Cada bloco corresponde a um slide de apresentacao.pdf e traz o horário em que ele deve começar e quanto tempo dura. O texto é um guia, não algo para ler palavra por palavra: fale com as suas palavras e use as frases em negrito como âncora.

[*Entre colchetes*] ficam gestos e ações. A **deixa** no fim de cada bloco é a frase de transição para o slide seguinte.

Relógio. Fundamentos até 10h22 · Problema 1 até 10h37 · Problema 2 até 10h55 · Perguntas até 11h00. Se atrasar, os slides 8, 12 e 35 podem ser passados rápido: o conteúdo deles está completo na apostila.

00 ABERTURA

10h00 – 10h03

SLIDE 1 · CAPA

10h00 · 1 min

Bom dia a todos. Obrigado por estarem aqui. Eu sou o Ronivaldo, estagiário de Análise e Desenvolvimento de Sistemas aqui na Marinha.

Nesta hora eu quero mostrar **duas coisas que resolvi no estágio**: como trazer um sistema que já existia para o GitLab da instituição, e como usei um agente de IA para documentar sistemas que não tinham documentação nenhuma. Para chegar lá, a gente passa antes pelos fundamentos de Git e GitLab.

→ **Deixa:** *Deixa eu mostrar o caminho que a gente vai fazer.*

SLIDE 2 · O QUE VAMOS VER NESTA HORA

10h01 · 2 min

São quatro partes. [*aponte para os números*] Primeiro, os **fundamentos**: Git, GitLab, branch, Merge Request e o GitLab pelo terminal. Depois, o **Problema 1**, que é integrar um sistema já versionado ao GitLab institucional. Em seguida, o **Problema 2**: documentar sistemas legados com IA. E fecho com a visão geral.

[*aponte para a caixa amarela*] Um aviso importante: **hoje é só exposição**. Não precisam anotar os comandos. Tudo o que eu mostrar está na apostila, com mais calma, passo a passo e com exercícios práticos para fazerem depois.

→ **Deixa:** *Então vamos começar pelo começo: por que versionar?*

01 FUNDAMENTOS

10h03 – 10h22

SLIDE 3 · DIVISÓRIA – FUNDAMENTOS

10h03 · 15 s

Primeira parte: fundamentos.

SLIDE 4 · POR QUE VERSIONAR?

10h03 · 1 min 30

[pausa, deixe o slide falar] Quem aqui nunca teve uma pasta assim? “sistema_v2_final_AGORA_VAI”.

O problema dessa pasta não é o nome feio. É que ela **não responde nada**: quem mudou, o quê, por quê, e qual é a versão certa. Agora imaginem isso com um sistema de milhares de arquivos e várias pessoas mexendo ao mesmo tempo.

[aponte para a coluna da direita] Um controle de versão responde exatamente essas perguntas: **o que, quem, quando e por quê**. Permite voltar a qualquer ponto e deixa várias pessoas trabalhando em paralelo.

→ **Deixa:** *E a ferramenta que faz isso é o Git.*

SLIDE 5 · O QUE É O GIT?

10h05 · 2 min

O Git é um **sistema distribuído de controle de versão**. Vou traduzir essa frase.

Cada **commit** é uma fotografia do projeto num momento, com quem tirou a foto, quando e por quê. *[aponte para A, B, C, D]* As fotos se encadeiam e formam o **histórico**. A etiqueta “main” é só um marcador apontando para a foto mais recente.

“Distribuído” quer dizer que **cada pessoa tem o histórico completo no próprio computador**. *[aponte para o código]* Esses três comandos criam um repositório e registram o primeiro commit sem servidor nenhum, sem internet.

→ **Deixa:** *Antes de seguir, um aviso honesto sobre o Git.*

SLIDE 6 · UM AVISO HONESTO

10h07 · 1 min

[deixe as pessoas lerem por alguns segundos] Essa tirinha é famosa entre desenvolvedores porque é verdade para muita gente: decora uns comandos e, se der erro, apaga tudo e baixa de novo.

A proposta de hoje é o contrário: **entender o modelo por trás dos comandos**. Se vocês entenderem commit, branch e merge, os comandos passam a fazer sentido e ninguém precisa apagar projeto.

→ **Deixa:** *E aí aparece a primeira confusão comum: Git, GitLab e GitHub.*

SLIDE 7 · GIT, GITLAB, GITHUB E GLAB

10h08 · 2 min

São quatro nomes parecidos para coisas diferentes.

O **Git** é o sistema de controle de versão, o que roda no computador. **GitLab** e **GitHub** são plataformas *em cima* do Git: hospedam o repositório e acrescentam revisão, permissões, automação. A diferença que mais importa para nós é que **o GitLab pode ser instalado na infraestrutura da própria instituição**. E o **glab** é a ferramenta de linha de comando do GitLab.

[aponte para a caixa amarela] Se ajudar a guardar: **o Git é o motor, o GitLab é a garagem com oficina e controle de acesso, e o glab é o controle remoto da garagem**.

→ **Deixa:** *Vamos ver o que tem nessa garagem.*

SLIDE 8 · GITLAB: UMA PLATAFORMA INTEIRA

10h10 · 1 min

Além de guardar o código, o GitLab reúne num só lugar: branches, Merge Requests, revisão de código, permissões, Issues para organizar tarefas, CI/CD para testar e publicar automaticamente, releases, e a parte de auditoria e segurança.

Não vamos ver tudo hoje. **O foco é branch e Merge Request**, que é o que usamos no dia a dia.

→ **Deixa:** *Mas antes, o ponto que interessa diretamente a uma instituição como a nossa.*

SLIDE 9 · GITLAB SELF-MANAGED

10h11 · 2 min

O GitLab tem uma versão que a instituição **instala e administra nos próprios servidores**. É o Self-Managed. *[aponte para a linha tracejada]* O código, o histórico e os usuários ficam do lado de dentro, sem ir para uma nuvem externa. A instituição controla quem acessa, as permissões e a auditoria.

[aponte para a caixa vermelha] Mas cuidado com uma conclusão errada: **estar dentro de casa não é o mesmo que estar seguro**. Segurança continua dependendo de configuração, atualização, backup, firewall, monitoramento e controle de acesso. O ganho do Self-Managed é o *controle*, não uma segurança automática.

→ **Deixa:** *Agora, dentro do GitLab, como o trabalho de várias pessoas se organiza? Com branches.*

SLIDE 10 · BRANCH

10h13 · 2 min

Uma branch é uma **linha de desenvolvimento**. *[aponte para a linha azul]* A main é a linha principal. *[aponte para a verde]* Alguém sai dela para criar o login, trabalha isolado e depois volta. *[aponte para a amarela]* Outra pessoa, ao mesmo tempo, corrige um bug em outra linha.

Isso permite trabalho isolado, em paralelo, com espaço para experimentar, e revisão antes de integrar.

[aponte para a caixa] Um detalhe técnico que ajuda muito: **branch não é uma cópia do projeto**. É só um ponteiro para um commit. Por isso criar uma branch é instantâneo, mesmo num sistema enorme.

→ **Deixa:** *E quando o trabalho numa branch termina, como ele volta para a main? Pelo Merge Request.*

SLIDE 11 · MERGE REQUEST

10h15 · 1 min 30

O Merge Request é um **pedido para integrar uma branch em outra**. No GitHub ele se chama Pull Request, é a mesma ideia.

[percorra os passos] A equipe visualiza as mudanças, comenta, pede ajustes, os testes automáticos rodam, alguém com permissão aprova, e aí acontece o merge.

Resumindo em três palavras *[aponte para a linha de baixo]*: **commit registra, branch isola, Merge Request pede a integração**.

→ **Deixa:** *Vou mostrar rapidamente os comandos que fazem isso.*

SLIDE 12 · OS COMANDOS DO DIA A DIA

10h16 · 1 min 30

[não leia a tabela inteira] Não precisam decorar, a tabela está na apostila. Só três observações.

À esquerda, o ciclo básico: `status` para ver o estado, `add` para preparar, `commit` para registrar. À direita, o trabalho com o servidor.

E uma diferença que confunde muita gente: **`fetch` baixa as novidades sem mexer nos seus arquivos; `pull` baixa e já mescla**. O `fetch` vai ser importante no Problema

1.

→ **Deixa:** Juntando esses comandos, o fluxo de uma equipe fica assim.

SLIDE 13 · FLUXO BÁSICO DE TRABALHO EM EQUIPE

10h18 · 1 min 30

[percorra da esquerda para a direita] Atualiza com `pull`, altera o código, confere com `status`, prepara com `add`, registra com `commit`, envia com `push`. Até aqui, tudo no seu computador. Depois, no GitLab: Merge Request, revisão e merge.

[aponte para a caixa verde] A regra de ouro: **comece sempre com `git pull` e trabalhe numa branch própria**. Quanto mais tempo longe da main, maior a chance do próximo assunto: conflito.

→ **Deixa:** Então, o que é um conflito?

SLIDE 14 · CONFLITO

10h19 · 2 min

Conflito acontece quando **duas branches mudaram a mesma linha** de formas diferentes. Numa, o nome virou João; na outra, Maria. O Git não tem como saber qual está certo.

[aponte para a caixa amarela] E aqui está o ponto: **conflito não é erro**. É o Git dizendo “não consigo decidir isso sozinho, decide você”.

[aponte para o código] Ele marca o arquivo assim: entre os sinais de menor fica a versão da branch em que você está, depois dos iguais fica a versão que está chegando. Você deixa o que deve ficar, apaga os marcadores, faz `add` e `commit`. Pronto.

→ **Deixa:** Por último nos fundamentos: dá para fazer a parte do GitLab sem sair do terminal?

SLIDE 15 · GITLAB CLI

10h21 · 1 min

Dá, com o **glab**. Você autentica uma vez e, a partir daí, cria Merge Request, lista, acompanha o pipeline, tudo pelo terminal.

[aponte para a caixa] Só um cuidado: **as regras do GitLab valem no terminal também**. Se a main exige aprovação, o glab não pula essa etapa.

→ **Deixa:** Com esses fundamentos, dá para entender o primeiro problema real.

02 PROBLEMA 1

10h22 – 10h37

SLIDE 16 · DIVISÓRIA – PROBLEMA 1

10h22 · 15 s

Problema 1: um sistema que já tinha histórico no computador precisa entrar no GitLab institucional, onde a main já existe e está protegida.

SLIDE 17 · O CENÁRIO NA INSTITUIÇÃO

10h22 · 1 min 30

Quando a gente pede um repositório no GitLab institucional, ele **já vem pronto**: com a branch main criada, um README padrão e a main protegida. Não dá para fazer push direto nela, só entra coisa por Merge Request.

Isso é ótimo para a governança. O problema é que os nossos sistemas **já existiam**, alguns com anos de histórico Git no computador. E a gente não queria perder esse

histórico.

→ **Deixa:** *E por que isso é um problema?*

SLIDE 18 · DOIS HISTÓRICOS QUE NÃO SE CONHECEM

10h24 · 2 min

[aponte para as duas linhas] Em cima, o histórico do sistema: A, B, C, D. Embaixo, o histórico que o GitLab criou: X e Y. As duas linhas **nasceram separadas**. Não existe nenhum commit em comum entre elas.

O Git só sabe juntar linhas que nasceram de um mesmo ponto. *[aponte para a mensagem]* Então, se você tenta um merge comum, ele simplesmente recusa: “refusing to merge unrelated histories”. E o push direto na main também é recusado, porque ela é protegida.

→ **Deixa:** *O que a gente quer, então?*

SLIDE 19 · O OBJETIVO

10h26 · 1 min

Quero chegar a isto: **um commit de merge, o M, que une as duas linhas** sem perder nenhum commit.

Depois do M, os dois históricos passam a ter um ancestral em comum. E aí, o Merge Request para a main flui normalmente, sem ser recusado.

→ **Deixa:** *São oito passos. Os três primeiros são preparação.*

SLIDE 20 · PASSOS 1 A 3

10h27 · 2 min

Passo 1: `git remote add origin` e a URL. Estou dizendo ao Git local onde fica o repositório no GitLab.

Passo 2: `git fetch origin`. Baixo o histórico do GitLab *sem mesclar nada*. Ele fica guardado como `origin/main`, uma cópia de leitura.

Passo 3: renomeio a minha main local para desenvolvimento. *[aponte para a caixa]* Por quê? Para não confundir a minha main com a main protegida do GitLab. Agora tenho duas coisas separadas e com nomes claros.

→ **Deixa:** *Agora vem o passo que realmente une os históricos.*

SLIDE 21 · PASSO 4: AUTORIZAR A UNIÃO

10h29 · 1 min 30

O merge, agora com o parâmetro `--allow-unrelated-histories`.

[aponte para as duas caixas] Esse parâmetro assusta, então quero deixar claro: ele **não** significa “ignore os conflitos”. Significa **“eu sei que esses históricos não têm ancestral comum, e autorizo o Git a tentar juntar”**.

E como os dois lados têm um README diferente, o resultado típico é um conflito no README.

→ **Deixa:** *E é resolvendo esse conflito que aparece a parte que mais causa erro.*

SLIDE 22 · PASSO 5: OURS E THEIRS

10h31 · 3 min

[fale devagar, este é o ponto central do bloco] Para resolver um conflito de um arquivo inteiro, o Git oferece dois atalhos: **ours**, “o nosso”, e **theirs**, “o deles”.

A armadilha é achar que “nosso” quer dizer “o arquivo do meu computador”. **Não é isso**. Os dois arquivos estão no meu computador. *[aponte para o diagrama]* **Ours**

é o lado da branch em que eu estou no momento do merge. **Theirs** é o lado que está chegando.

No nosso caso, eu estou na branch desenvolvimento, que tem o README do sistema. Eu queria manter esse README. Então o comando é `git restore --ours README.md`. Se eu quisesse o README institucional, seria `--theirs`.

[aponte para a caixa vermelha] Se interpretar ao contrário, você preserva o arquivo errado e nem percebe.

→ **Deixa:** Decidido o conflito, falta registrar e publicar.

SLIDE 23 · PASSOS 6 E 7

10h34 · 1 min

Passo 6: `add` e `commit`. Esse commit é o M, o que une os dois históricos.

Passo 7: `git push origin desenvolvimento`. *[aponte para a direita]* No GitLab passam a existir, lado a lado, a main intocada e protegida, e a branch desenvolvimento com o sistema e o histórico integrado.

→ **Deixa:** E o último passo fecha o ciclo.

SLIDE 24 · PASSO 8: O MERGE REQUEST

10h35 · 1 min 30

Abro um Merge Request de desenvolvimento para main. Pode ser pela interface web, com a aprovação de quem tem permissão no repositório, ou pelo terminal com o `glab mr create`.

[aponte para a caixa verde] E agora ele **não é recusado**, porque os históricos são relacionados e todo conflito já foi tratado no computador. Esse é o ponto final e fundamental do processo.

→ **Deixa:** Resumindo tudo em uma tela.

SLIDE 25 · RESUMO EM OITO LINHAS

10h36 · 1 min

[percorra os comentários à direita] Conectar, baixar, renomear, unir, decidir o conflito, registrar, publicar e pedir a integração. Oito linhas.

[aponte para a caixa] Na apostila tem um exercício que **reproduz esse caso inteiro no computador de vocês**, sem precisar de servidor. Recomendo muito: é o melhor jeito de fixar.

→ **Deixa:** Com isso, os sistemas passaram a estar versionados. Mas surgiu outro problema.

03 PROBLEMA 2

10h37 – 10h55

SLIDE 26 · DIVISÓRIA – PROBLEMA 2

10h37 · 15 s

Problema 2: os sistemas agora estão no GitLab. Mas quase nenhum tinha documentação.

SLIDE 27 · VERSIONADO NÃO É DOCUMENTADO

10h37 · 1 min 30

Ter histórico não significa que alguém *entende* o sistema. *[percorra a cadeia]* Código existente com pouca documentação leva a conhecimento concentrado em poucas pessoas. Isso deixa manutenção e evolução difíceis e, no fim, vira **risco operacional**:

se a pessoa que conhece sai, o conhecimento vai junto.

[aponte para a caixa] A solução que adotei foi usar um **agente de IA**, o Claude Code, para acelerar o entendimento, a refatoração e a documentação.

→ **Deixa:** *Mas é importante dizer como a IA entra nisso.*

SLIDE 28 · IA: FERRAMENTA, NÃO AUTORIDADE

10h39 · 2 min

A ideia **não é “mandar a IA escrever a documentação”**. É usá-la como ferramenta de engenharia.

[percorra da esquerda para a direita] O engenheiro define o objetivo e as restrições. O agente analisa, sugere, documenta e só executa o que foi autorizado. E no fim o engenheiro revisa, valida no sistema real e decide.

[aponte para a caixa vermelha] A IA pode errar: interpretar mal uma regra de negócio, a intenção de quem escreveu o código. Por isso **toda documentação gerada por IA é validada contra o sistema real**. A decisão é sempre humana.

→ **Deixa:** *E aqui entra uma parte que eu acho a mais interessante: como o prompt evoluiu.*

SLIDE 29 · COMO O PROMPT EVOLUIU

10h41 · 3 min

O prompt é a instrução que a gente dá ao agente. *[aponte para a fonte, embaixo]* Esses dados são reais: vêm do histórico de uso da ferramenta, com 717 prompts em 123 sessões desde julho.

[percorra as barras de cima para baixo]

- **V0**, em julho: já pedia para não modificar nada e descobrir antes de escrever.
- **V1**: pedidos curtos em português, que cresciam a cada vez que eu reusava.
- **V2**: uma lista estruturada com 47 itens.
- **V3**: um protocolo grande, em inglês, com uma fase só de leitura.
- **V4**, o final: 42 seções, com saída obrigatória em português.

Reparem na escala: ela é logarítmica. **Do primeiro pedido livre ao final, o prompt cresceu cerca de 164 vezes**. E o V4 é menor que o V3: ficou mais enxuto e mais rigoroso ao mesmo tempo.

→ **Deixa:** *E esse crescimento não foi à toa: cada regra nova veio de uma falha.*

SLIDE 30 · CADA FALHA VIROU UMA REGRA

10h44 · 2 min

[leia linha a linha, da esquerda para a direita] A documentação saía superficial, e eu tinha que pedir de novo: entrou a análise em fases. O agente preenchia lacunas sozinho: entrou a regra de registrar como GAP e nunca inventar. Havia risco de alterar código durante a análise: entrou a fase zero, só leitura. O prompt em inglês gerou resposta em inglês: entrou a exigência de tudo em português. E o agente seguia em frente diante de risco: entrou o protocolo de parar e reportar.

A mensagem é: **um bom prompt não nasce pronto. Ele é o registro das falhas que você já corrigiu.**

→ **Deixa:** *E isso melhorou o resultado? Os números.*

SLIDE 31 · O QUE MELHOROU, EM NÚMEROS

10h46 · 2 min

Com os prompts livres, **cerca de 53% das tarefas de documentação precisaram de retrabalho**: refazer, reenviar, pedir para continuar de onde parou. Com os prompts estruturados, com contrato de execução, **caiu para cerca de 25%**.

[aponte para a caixa amarela, com honestidade] Quero ser transparente: é uma **estimativa com amostra pequena**. Foram 15 tarefas de um lado e 4 do outro, e parte das paradas foi por limite de uso da ferramenta, não do prompt. Não é prova estatística, é indicação. O que é concreto são as cinco regras nascidas de cinco falhas.

→ **Deixa:** Como o prompt final organiza o trabalho? Em fases.

SLIDE 32 · O MÉTODO EM FASES

10h48 · 2 min

O prompt proíbe fazer tudo de uma vez. *[percorra as fases]* Fase zero: descoberta só de leitura. Depois o mapa do sistema, a análise profunda, um modelo de conhecimento, e, antes de documentar, **uma fase só para validar o entendimento**. Aí sim vem a documentação, e depois auditoria e evolução.

[aponte para a caixa] Por quê? Para evitar a cascata: **entendimento errado gera documentação errada, que gera recomendação errada, que gera um projeto futuro errado**. Em vez disso: descobrir, evidenciar, modelar, validar, documentar, auditar e evoluir.

→ **Deixa:** E dentro dessas fases, o prompt tem três travas de segurança.

SLIDE 33 · AS TRÊS TRAVAS DE SEGURANÇA

10h50 · 2 min 30

[uma caixa de cada vez] **Primeira: somente leitura**. Na descoberta o agente não altera, não apaga, não renomeia, não faz commit. Descobrir antes de modificar.

Segunda: nunca inventar. Todo fato precisa de evidência no código. O que não dá para confirmar recebe um rótulo: desconhecido, não verificado, inferido, requer validação. E o prompt separa como o sistema é hoje, como *deveria ser*, e a diferença entre os dois.

Terceira: parar e reportar. Diante de dúvida, contradição, risco ou falta de autorização, o agente **para, reporta e aguarda uma decisão humana**. E parar não é falhar: é o comportamento certo.

→ **Deixa:** Uma última observação sobre o prompt: o idioma.

SLIDE 34 · PROMPT EM INGLÊS, DOCUMENTAÇÃO EM PORTUGUÊS

10h52 · 1 min 30

O prompt final está em inglês, e a documentação sai em português. **Idioma do prompt e idioma do resultado são coisas diferentes**.

Isso eu aprendi na prática: *[aponte para os itens]* quando o V3 foi só em inglês, o agente passou a responder em inglês. No dia seguinte, o V4 ganhou a regra *[aponte para o texto em azul]*: todo conteúdo em português brasileiro.

[aponte para a nota de rodapé] A motivação do inglês é economizar tokens, mas isso eu não medi, então fica como prática adotada, não como resultado.

→ **Deixa:** E, afinal, o que esse prompt entrega?

SLIDE 35 · O QUE O PROMPT PEDE E ENTREGA

10h53 · 1 min 30

Ele se resume a seis perguntas: *[percorra a tabela]* o que existe, como as partes se relacionam, como funciona, quais os riscos, como virar documentação útil, e se ela bate com o sistema real.

[aponte para a árvore de pastas] E a entrega é uma pasta de documentação: descoberta, mapa, arquitetura, regras de negócio, banco, APIs, segurança, problemas conhecidos, dívida técnica e um guia para leigos.

O objetivo é transformar “**código que poucas pessoas conhecem**” em “**sistema que outras pessoas entendem e mantêm**”.

→ **Deixa:** *Vamos juntar as duas histórias.*

04 ENCERRAMENTO

10h55 – 11h00

SLIDE 36 · DIVISÓRIA – ENCERRAMENTO

10h55 · 10 s

Para fechar.

SLIDE 37 · A VISÃO GERAL

10h55 · 1 min

Os dois problemas parecem diferentes, mas se completam. *[aponte para a esquerda]* De um lado, **versionamento**: Git e GitLab dão histórico e revisão. *[aponte para a direita]* Do outro, **conhecimento**: IA com validação humana gera documentação. *[aponte para baixo]* Os dois juntos levam ao mesmo lugar: manutenção e evolução seguras.

SLIDE 38 · MENSAGEM FINAL

10h56 · 1 min 30

O conhecimento técnico de uma equipe **não pode morar só na cabeça de quem escreveu o sistema**.

O código precisa ter histórico. O sistema precisa ter documentação. As alterações precisam de revisão. As decisões precisam de contexto.

E a IA pode acelerar muito esse trabalho, **desde que dentro de um processo controlado**: objetivo claro, restrições bem definidas e validação humana.

[pausa] No fim, o objetivo é um só: **tornar os sistemas mais fáceis de compreender, manter, compartilhar e evoluir**.

SLIDE 39 · OBRIGADO E PERGUNTAS

10h57 · 3 min

Muito obrigado pela atenção. *[aponte para o texto de baixo]* A apostila traz tudo o que vimos, com mais calma, o passo a passo para reproduzir o Problema 1 e os exercícios práticos.

Fico à disposição para perguntas.

[se sobrar tempo e ninguém perguntar, puxe uma destas:]

- “Alguém já passou por um conflito de merge? Como resolveu?”
- “Qual sistema aqui vocês acham que mais precisaria de documentação?”

PERGUNTAS PROVÁVEIS

respostas curtas

Posso usar IA com o código dos sistemas da Marinha?

Só com autorização e dentro das normas de segurança da informação da OM. O agente envia trechos do código para um serviço externo. Para praticar, usem projetos pessoais ou de código aberto.

E se eu fizer o merge errado?

Enquanto o merge não virou commit, `git merge --abort` desfaz tudo. Depois do commit, dá para desfazer com `git revert`. Nada se perde no Git sem querer.

Por que não usar `push --force` na main?

Porque ela é protegida justamente para não aceitar isso, e porque apagaria o histórico institucional. O caminho correto é o Merge Request.

A IA não inventa coisas?

Pode inventar. Por isso o prompt exige evidência para cada fato, rótulos para o que não se confirma, e validação humana no fim.

Rebase funciona para o Problema 1?

Até funciona, mas reescreve o histórico e inverte o sentido de `ours` e `theirs`. O merge preserva tudo como aconteceu, e é mais seguro para quem está começando.

Onde está o prompt completo?

No arquivo `model/SYSTEM_DOCUMENTATION.md`, junto com a apostila. O capítulo 6 da apostila explica cada parte.